Redes de Datos

Clase 2

Universidad Tecnológica Nacional - FRH

Técnicas Digitales III

Ciclo Lectivo 2020

Autor: Alejandro Furfaro / Mariano Mercado

Sockets (1)

Creación de un socket.

SINOPSIS
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int socket (int dominio, int tipo, int protocolo);

Socket crea un extremo de una comunicación y devuelve un descriptor.

El parámetro **dominio** especifica un dominio de comunicaciones dentro del cual tendrá lugar la comunicación; esto selecciona la familia de protocolo que deberá emplearse. Estas familias se definen en el fichero de cabecera sys/socket.h.
Los formatos actualmente reconocidos son:

AF_UNIX (protocolos internos de UNIX)
AF_INET (protocolos de ARPA Internet)
AF_ISO (protocolos ISO)
AF_NS (protocolos de Xerox Network Systems)
AF_IMPLINK (capa de enlace IMP "anfitrión en IMP")

Sockets (2)

El campo **tipo** especifica la semántica de la comunicación. Los tipos definidos en la actualidad son:

SOCK_STREAM → **TCP**
SOCK_DGRAM → **UDP**
SOCK_RAW
SOCK_SEQPACKET
SOCK_RDM

Un socket SOCK_STREAM proporciona flujos de bytes basados en una conexión bidireccional secuenciada, confiable. (TCP)

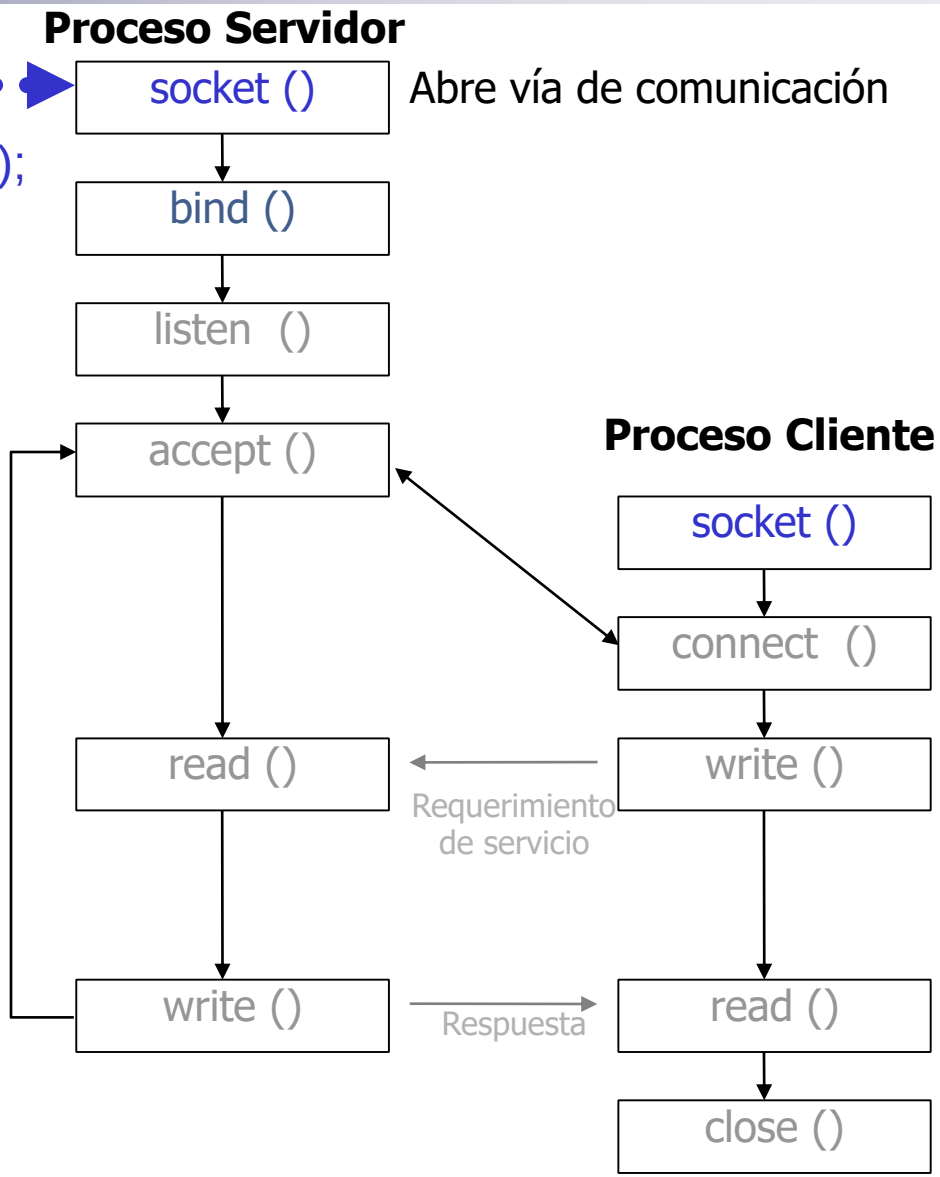
Un socket SOCK_DGRAM admite datagramas: mensajes no confiables, sin conexión, de una longitud máxima (normalmente pequeña). (UDP)

El campo **protocolo** especifica un protocolo particular para ser usado con el socket. Normalmente por cada "tipo" de socket existe un solo protocolo, por lo cual **este argumento se completa con 0.** En el caso que existieran mas de uno, con este argumento se selecciona el protocolo deseado.

Sockets (3)

Creación

```
int     socket (familia, tipo, Protocolo);
int     familia;
int     tipo;
int     Protocolo;
```



Sockets (4) – bind ()

SINOPSIS

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
int bind(int sockfd, struct sockaddr * direccion, int tam);
```

bind asocia al socket un número de puerto y su dirección IP.

Sockfd: File descriptor devuelto por socket().
Direccion: Variable tipo struct sockaddr (Para la familia AF_INET se utiliza la estructura sockaddr_in:

```
struct sockaddr_in
        {
        short sin_family;           /*AF_INET*/
        u_short     sin_port;       /* Número de port */
        struct in_addr sin_addr;    /*Identificación dentro de  la red y host */
        char sin_zero [8];          /* cero el resto de la estructura */
        };

struct in_addr
        {
        u_long s_addr;
        };
```

my_addr tiene una longitud de addrlen en bytes.

Sockets (5) – bind ()

Ejemplo de socket y bind:

```
#define MYPORT 4321    // puerto escucha UDP

//preparo socket

        if ((sockfd = socket ( AF_INET, SOCK_DGRAM, 0 ) ) == -1)  /* Socket UDP */
                {
                perror("socket");
                exit(1);
                }
        my_addr.sin_family = AF_INET;                   /* host byte order */
        my_addr.sin_port = htons (MYPORT);              /* short, network byte order */
        my_addr.sin_addr.s_addr = INADDR_ANY;           /* auto-fill with my IP */
        bzero( & (my_addr.sin_zero), 8);                /* zero the rest of the struct */

        if (bind (sockfd, (struct sockaddr *) & my_addr, sizeof(struct sockaddr)) == -1)
                {
                perror("bind");
                exit(1);
                }
```

Sockets (6)

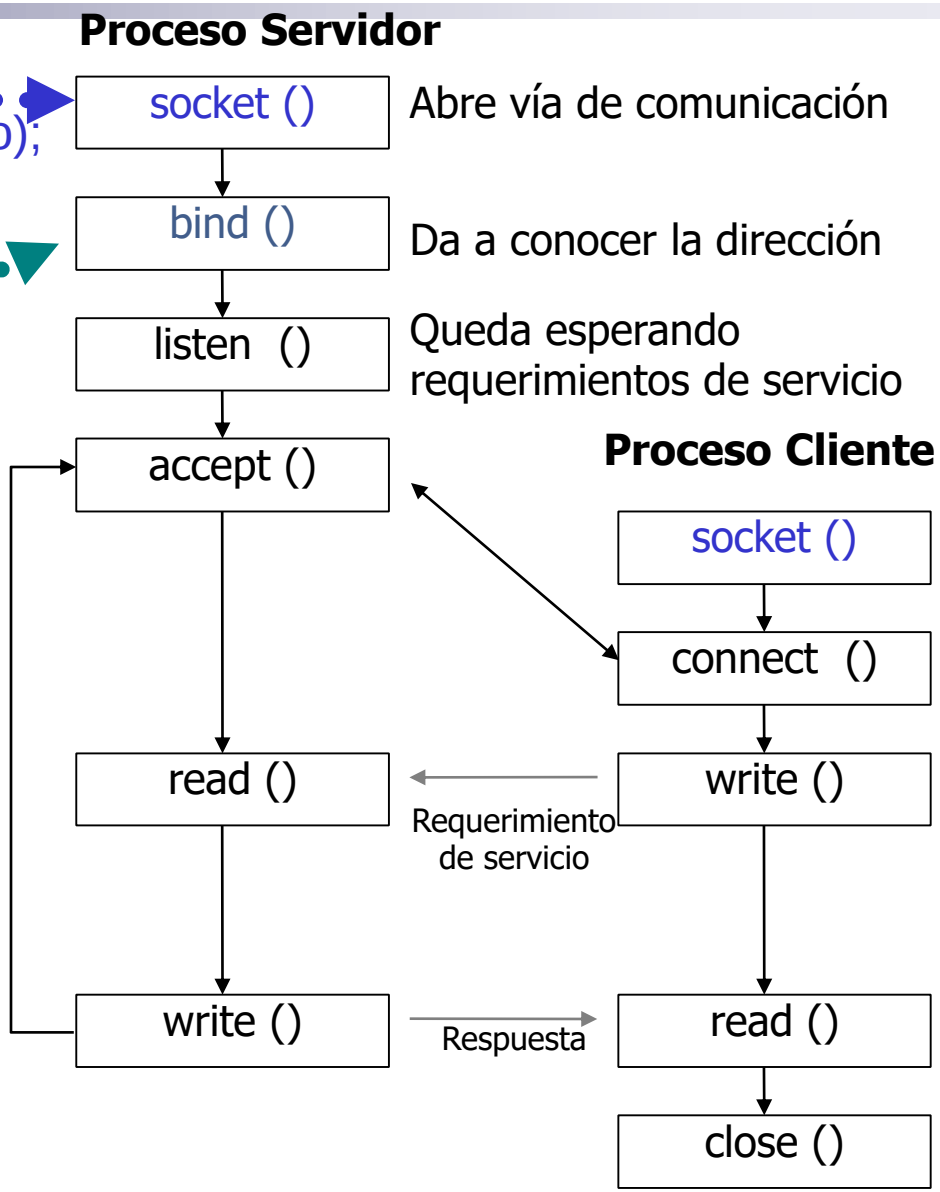
Creación

```
int     socket (familia, tipo, Protocolo);
int     familia;
int     tipo;
int     Protocolo;
```

Enlace (Bind)

```
int  bind (sock, direccion, tam)
int     sock;
struct sockaddr * direccion;
int     tam;
```

```
struct sockaddr_in
{
short sin_family;
u_short    sin_port;
struct in_addr sin_addr;
char sin_zero [8];
};
```



Sockets (7)

- **Listo**

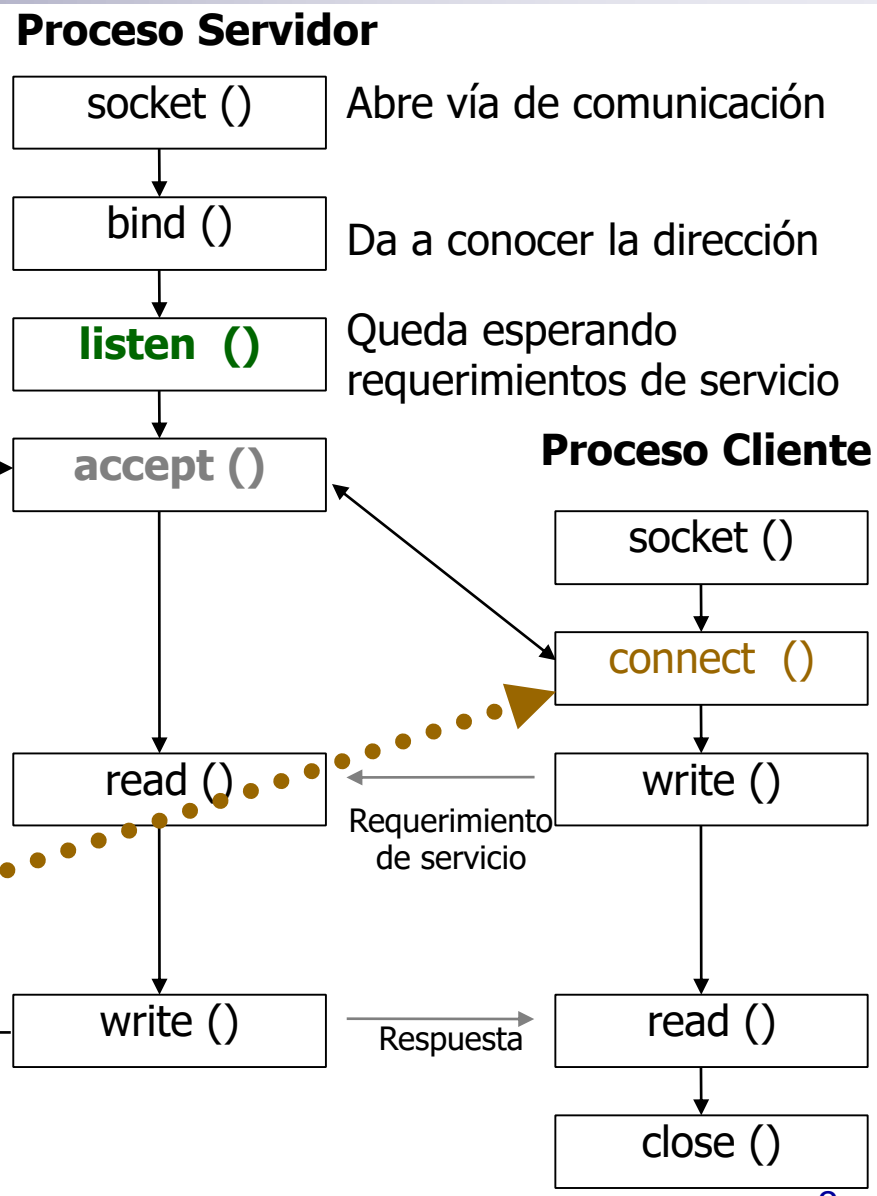
```
int listen (sock, buffer);
int   sock;
int   buffer;
```

- **Aceptada**

```
int accept (fd, direccion, tam);
int   fd;
void* direccion;
int   tam;
```

- **Conectando**

```
int   connect  (fd, direccion, tam );
int   fd;
struct          sockaddr *direccion;
int   tam;
```



Sockets (8) – listen ()

SINOPSIS
#include <sys/socket.h>
#include <netinet/in.h>

int listen (int sockfd, int buffer);

LISTEN prepara a un socket para recibir conexiones.

Para aceptar conexiones, primero se crea un socket con "socket", luego se especifica con "listen" el deseo de aceptar conexiones entrantes, y un límite de la cola para dichas conexiones, y por último las conexiones son aceptadas mediante "accept". La llamada listen se aplica solamente a sockets del tipo SOCK_STREAM (TCP – Orientados a la conexión).

El parámetro buffer define la longitud máxima a la que puede llegar la cola de conexiones pendientes.

Una conexión sale de la cola de conexiones pendientes cuando es aceptada.

Sockets (9) – accept ()

SINOPSIS
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
int accept (int sockfd, struct sockaddr * addr, int * addrlen);

Accept, acepta una conexión sobre un socket. (Solo del tipo TCP).

El argumento sockfd es un socket fd que ha sido creado con "socket", ligado a una dirección con "bind" y que se encuentra a la escucha tras un "listen". La función accept extrae la primera petición de conexión de la cola de conexiones pendientes, **crea un nuevo socket con las mismas propiedades de sockfd y asigna un nuevo descriptor para el socket**.
Si no hay conexiones pendientes en la cola y el socket no está marcado como "no bloqueante", accept se bloquea hasta que se presente una conexión.

El argumento addr es un parámetro de salida al que se asigna los datos del otro extremo conectado. El formato exacto del parámetro addr viene determinado por el dominio en el que se produce la comunicación. Para AF_INET la estructura es del tipo sockaddr_in.

Finalmente addrlen es un parámetro de entrada/salida; al efectuar la llamada debe contener la cantidad de espacio apuntado por addr; a la salida, contendrá la longitud real (en bytes) de la dirección devuelta.

Sockets (10) – connect ()

SINOPSIS
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

int connect (int sockfd, struct sockaddr *serv_addr, int addrlen);

Connect, inicia una conexión en un socket

El argumento sockfd es un socket fd que ha sido creado con "socket".
El argumento addr es un parámetro al cual se asigna los datos del otro extremo al cual nos queremos conectar. El formato exacto del parámetro addr viene determinado por el dominio en el que se produce la comunicación. Para AF_INET la estructura es del tipo sockaddr_in.

Finalmente addrlen es un parámetro que debe contener el tamaño de la estructura sockaddr_in.

Sockets (11)

- **Lectura**

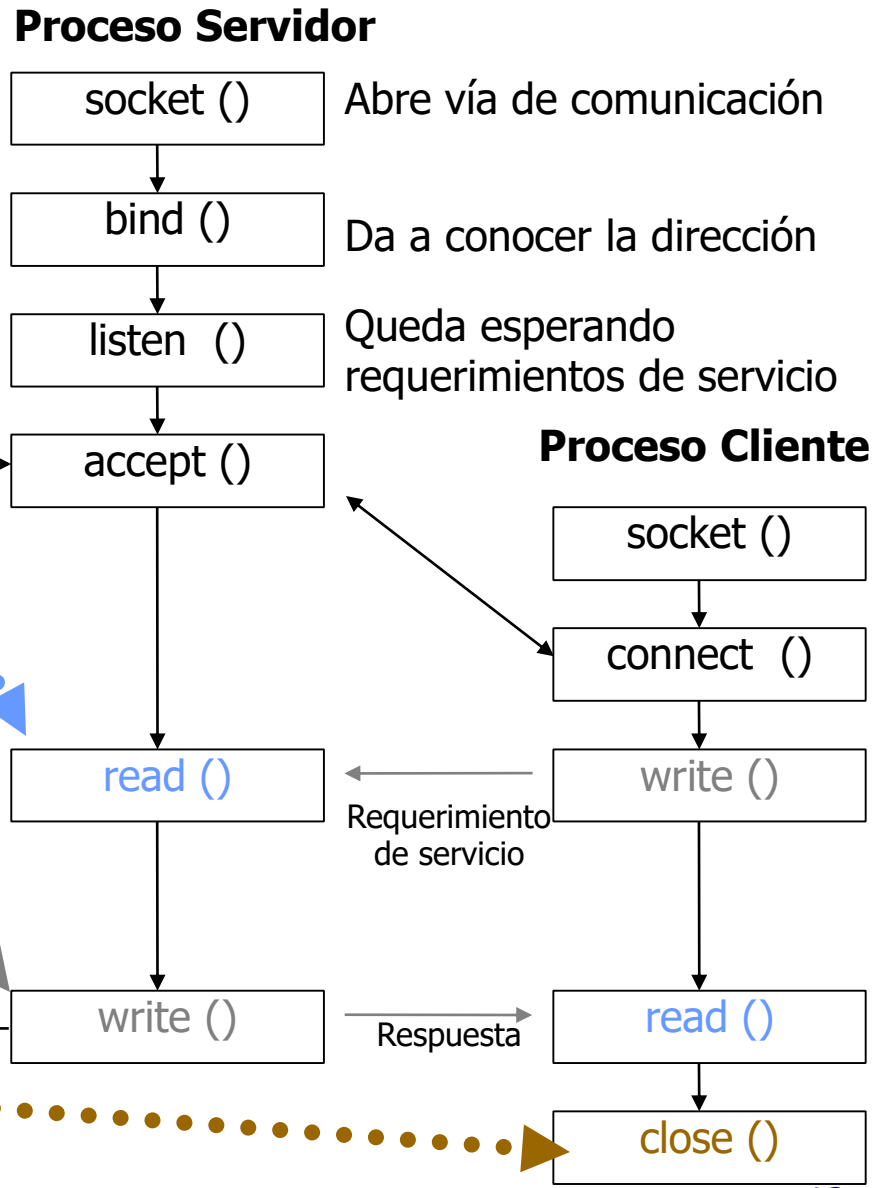
```
Int   read (fd , array, tam);
Int   fd;
char *array;
int   tam;
```

- **Escritura**

```
int write (fd, array, tam);
int   fd;
void *array;
int   tam;
```

- **Cierre**

```
int   close  (fd);
int   fd;
```



Sockets (12) - Otras Funciones de Lectura

- ssize-t readv (fd, array, tam);
  int fd;
  const struct iovec *array;
  ssize_t tam;
- int recv (fd, puntero, tam, control)
  int fd;
  void *puntero;
  int tam;
  int control;
- int recvmsg (fd, msg, control)
  int fd;
  struct msghdr msg[];
  int control;

```
Struct    iovec
{
caddr_t iov_base;/* Puntero al comienzo del espacio
reservado */
int iov_len;/* Tamaño de memoria reservada)*/
};
```

```
struc     msghdr
{
caddr_t   msg_Name;
int       tam_msg_Name; /* Estructura con los
datos. Ya fue vista */
struct iovec *msg_iov; /*Numero de elementos
de  msg_iov */
int       tam_msg_iov;  /* Flags de acceso */
caddr_t   msg_accrights;/* Tamaño de
msg_accrights */
int       msg_accrightslen;
};
```

- **int recvfrom (fd, puntero, tam, flags, origen, tam_origen)**
  int fd;
  void * puntero;
  int tam;
  int flags;
  struct sockaddr *origen;
  int * tam_origen;

Sockets (13)

Otras Funciones de Escritura

- ssize-t writev (fd, array, tam);
  - int fd;
  - const struct iovec *array;
  - ssize_t tam;

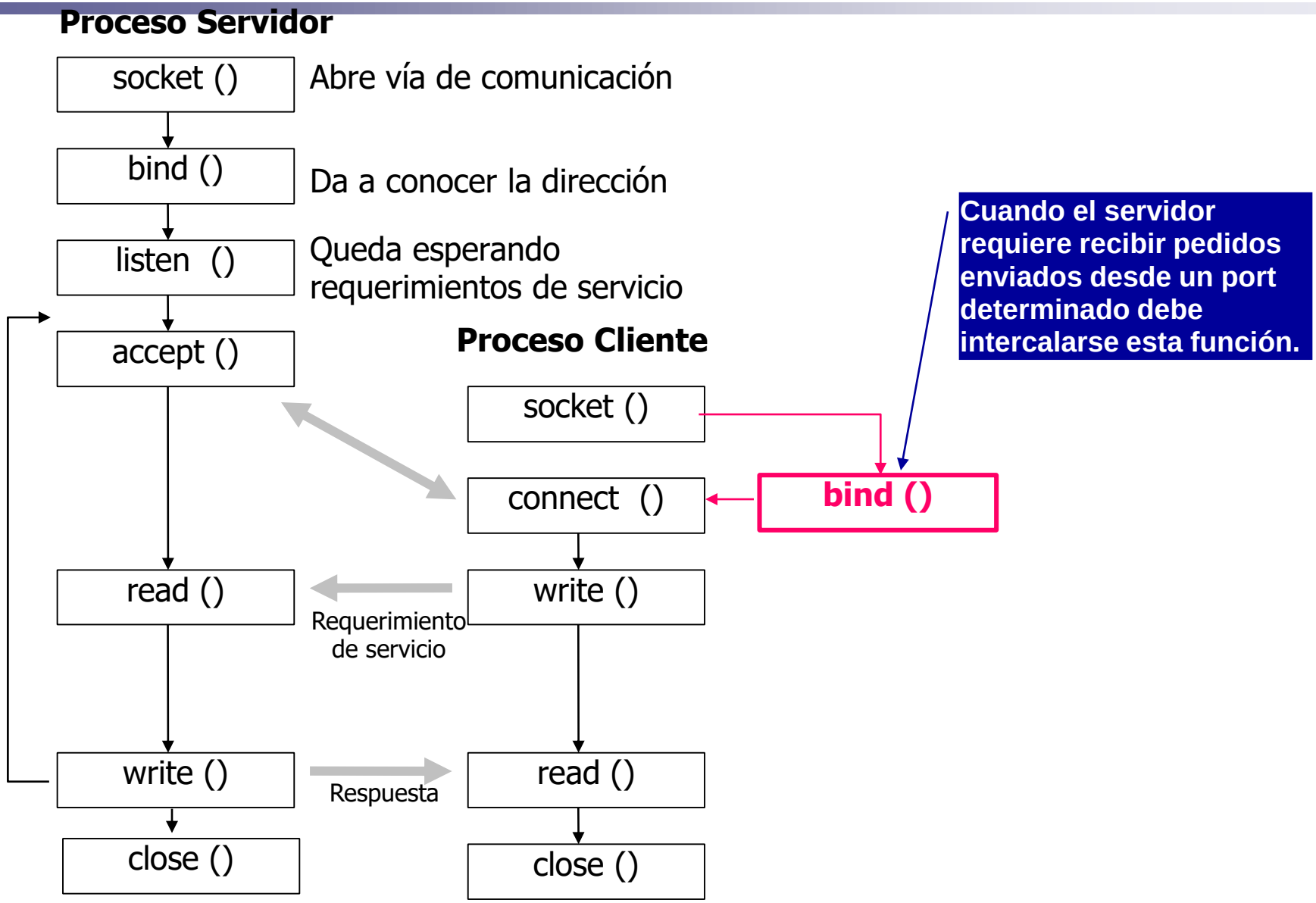
```
Struct      iovec
{
caddr_t iov_base; /* Puntero al comienzo del espacio reservado*/
int iov_len;/* Tamaño de memoria reservada)*/
};
```

- **int sendto (fd, puntero, tam, flags, destino, tam_destino);**
  - int fd;
  - void *puntero;
  - int tam;
  - int flags;
  - void *destino;
  - int * tam_destino;

```
struc       msghdr
{
caddr_t     msg_Name;
int         tam_msg_Name; /* Estructura con los datos. Ya fue vista */
struct iovec *msg_iov; /*Numero de elementos de  msg_iov */
int         tam_msg_iov;  /* Flags de acceso */
caddr_t     msg_accrights;/* Tamaño de msg_accrights */
int         msg_accrightslen;
};
```

- int sendmsg (fd, msg, control);
  - int fd;
  - struct msghdr msg[];
  - int control;
- int send (fd,puntero,tam,control);
  - int fd;
  - void * puntero;
  - int tam;
  - int control;

Sockets (14) - TCP



Sockets (14) - UDP

